JPQL:Instructions:
A) Create an employeeTable table with the following fields: empId, empFirstName, empLastName, empSalary, empAge.
B) Create an Employee entity having following fields: id, firstName, lastName, salary, age which maps to the table columns given in above.
Questions:
1) Display the first name, last name of all employees having salary greater than average salary ordered in ascending by their age and in descending by their salary.
2) Update salary of all employees by a salary passed as a parameter whose existing salary is less than the average salary.
3) Delete all employees with minimum salary.

```java
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name="employeeTable")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "empId")
    private Long id;
    @Column(name = "empFirstName")
    private String firstName;
    @Column(name = "empLastName")
    private String lastName;
    @Column(name = "empSalary")
    private Double salary;
    @Column(name = "empAge")
    private Integer age;
```

1)

```
@Repository  2 usages
public interface EmployeeRepo extends JpaRepository<Employee, Long> {

    @Query("select emp.firstName as firstName, emp.lastName as lastName " +  1 usage
            "from Employee  as emp" +
            " where emp.salary > (select AVG(e.salary) from Employee e)" +
            " order by emp.age asc, emp.salary desc")
    List<EmployeeDto> getEmployeeBySalaryAvg();
//
```

2)

```
//  2) Update salary of all employees by a salary passed as a parameter whose existing salary is less
@Modifying  1 usage
@Transactional
@Query("UPDATE Employee e " +
        "SET e.salary = :newSalary " +
        "WHERE e.salary < (SELECT AVG(emp.salary) FROM Employee emp)")
int updateSalaryLessThanAverage(Double newSalary);
```

3)

```
    @Modifying  no usages
    @Transactional
    @Query("DELETE FROM Employee e " +
            "WHERE e.salary = (SELECT MIN(emp.salary) FROM Employee emp)")
    int deleteEmployeeWithMinSalary();

}
```

2)Native Query:
Questions:
1) Display the id, first name, age of all employees where last name ends with "singh".
2) Delete all employees with age greater than 45(Should be passed as a parameter).

```
    public List<Employee> findSinghEmp(){  1 usage
        return repo.findEmployeeByLastNameStartsWith("singh");
    }

    Rename usages
    public int deleteEmpGreaterThan(){
        return repo.deleteEmployeeByAgeGreaterThan( ageIsGreaterThan: 45);
    }
}
```

Typo: In word 'singh'
Replace with 'sing' Alt+Shift+Enter More actions... Ctrl+.

Service layer

```
    // 1) Display the id, first name, age of all employees where last name ends with "singh".
    List<Employee> findEmployeeByLastNameStartsWith(String lastName);  1 usage

    // Q)2 Delete all employees with age greater than 45(Should be passed as a parameter).
    int deleteEmployeeByAgeGreaterThan(Integer ageIsGreaterThan);  no usages

}
```

Repo Layer

3)
Inheritance Mapping:

1) Implement and demonstrate Single Table strategy.
2) Implement and demonstrate Joined strategy.
3) Implement and demonstrate Table Per Class strategy.

1)

```java
@AllArgsConstructor
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "emp_type")
public abstract class EmployeeModel {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}
```

Only one table created

Some fields null

Discriminator column

2)

```java

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Entity
//@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
//@DiscriminatorColumn(name = "emp_type")
@Inheritance(strategy  = InheritanceType.JOINED)
public abstract class EmployeeModel {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}

```

Multiple tables Created
Foreign key relationship
No null values

3)

```java
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Entity
//@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
//@DiscriminatorColumn(name = "emp_type")
//@Inheritance(strategy  = InheritanceType.JOINED)

@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)

public abstract class EmployeeModel {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}
```

Separate tables
No joins
Duplicate parent fields

Component Mapping:

1) Implement and demonstrate Embedded mapping using employee table having following fields: id, firstName, lastName, age, basicSalary, bonusSalary, taxAmount, specialAllowanceSalary.

```java
@AllArgsConstructor
@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;
    private Integer age;

    @Embedded
    private Salary salary;
}
```

```java
@Getter  no usages
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Embeddable
public class Salary {

    private Double basicSalary;
    private Double bonusSalary;
    private Double taxAmount;
    private Double specialAllowanceSalary;
}
```